Mitigating Malicious Code Generation in LLM Agents via Automated Peer Review and Sandboxing

Vinayak Pawar, Srajan Dwivedi
Department of Computer Science and Engineering, Amity University Gwalior, India - 2026

Abstract— Large language model (LLM) agents are increasingly used as autonomous programming assistants, but their ability to write and execute code creates a security risk when generated programs contain malicious behavior, unsafe APIs, or prompt-injected logic. This paper presents SafeCode-Agent, a zero-trust dual-agent architecture that separates code generation from independent security review. The framework combines a generator agent, an AST-based gatekeeper, a semantic monitor agent, and sandboxed execution to reject unsafe Python programs before deployment. Evaluation on a curated benchmark covering injection, privilege escalation, data exfiltration, obfuscation, and forbidden API misuse shows a 91.7% vulnerability detection rate and a 94.3% malicious script catch rate, substantially outperforming a single-agent baseline while adding 2.5 s average latency. The results indicate that layered peer review and containment can materially improve the safety of agentic code generation workflows.

Index Terms— LLM agents, code security, automated peer review, sandboxing, prompt injection, multi-agent systems, AST analysis.

I. INTRODUCTION

Autonomous LLM software agents have moved from code-completion tools toward systems that plan, edit files, call command-line tools, and execute generated programs. This shift improves productivity, but it also changes the security model: a model output is no longer merely text, but an operational artifact that may touch files, networks, credentials, or build systems.

Current agent pipelines often rely on the same model family to produce code and judge its own output. That arrangement creates correlated failure modes, especially under prompt injection, obfuscated payloads, and hallucinated API usage. Security checks must therefore be independent, layered, and capable of observing both static structure and runtime behavior.

This paper proposes SafeCode-Agent, a dual-agent verification framework for Python code generation. The design separates generation from review and enforces four stages: code synthesis, AST-based policy checks, semantic review by a monitor agent, and sandboxed execution. The contribution is a compact defense-in-depth architecture and an initial benchmark evaluation against high-risk code generation tasks.

The motivation for the framework is practical rather than purely theoretical. In classroom projects, hackathon prototypes, and early enterprise copilots, developers increasingly ask agents to generate utility scripts, data-processing tools, API wrappers, and deployment helpers. These programs are often run immediately, sometimes before a human reviewer has time to inspect every imported module or execution path.

A malicious or unsafe generated script may not look like malware in the traditional sense. It may read a sensitive path while claiming to collect configuration, use subprocess calls for convenience, change file permissions while installing dependencies, or embed an encoded payload inside an otherwise harmless helper. These gray-zone behaviors make simple keyword filtering insufficient and motivate a combination of syntactic, semantic, and runtime controls.

The central research question is whether independent automated peer review can reduce dangerous code generation without making agentic programming unusably slow. SafeCode-Agent answers this by treating generated code as untrusted input and requiring multiple independent approvals before execution or deployment.

II. BACKGROUND AND RELATED WORK

A. LLM Code Generation and Security Risks

LLMs trained on large code corpora can synthesize useful programs, but their outputs may contain vulnerable constructs, unsafe deserialization, shell command injection, or unbounded file access. These risks are amplified when agents receive tool access and can execute their own code.

Traditional code generation evaluations emphasize functional correctness, pass rates on programming benchmarks, or developer productivity. Those metrics are important but incomplete for autonomous agents because a program can satisfy the visible task while also violating security policy. For example, a generated script may correctly parse a log file while silently transmitting its contents, or it may solve an installation task by executing arbitrary shell commands.

The risk is also shaped by distribution shift. Models trained on public repositories observe many insecure idioms that were historically accepted in quick scripts: string-built SQL queries, unchecked shell invocations, broad exception handlers, and unrestricted filesystem traversal. When such patterns are reproduced by an agent, they can become operational vulnerabilities rather than merely low-quality code.

B. Prompt Injection and Agentic Attack Surface

Prompt injection is especially concerning for agentic systems because malicious instructions can be embedded in user inputs, repository files, comments, or generated code. A reviewer that evaluates comments and docstrings naively may be manipulated by adversarial natural language rather than program semantics.

Unlike static prompts sent to a chatbot, agent prompts evolve during task execution. The agent may read README files, issue tracker comments, downloaded data, or intermediate

code generated by another model. Each artifact can carry instructions that compete with the developer's original intent. A security monitor must therefore avoid trusting natural-language material inside code comments and must privilege program behavior over persuasive text.

Prompt injection also interacts with tool permissions. A model that can call a shell, edit files, or access network utilities can transform an injected instruction into concrete side effects. The SafeCode-Agent design assumes that any generated code may contain adversarial instructions and that the monitor must evaluate the code under a hostile-input assumption.

C. Static Analysis, Semantic Review, and Sandboxing

Static analysis provides deterministic rejection of known-dangerous imports, functions, and syntax patterns. Semantic review complements this by identifying intent, indirect misuse, and suspicious control flow. Sandboxing adds the final containment layer by executing code with resource limits and telemetry capture before deployment approval.

Each layer covers a different failure mode. Static analysis is fast and reproducible but struggles with intent and with obfuscation that is syntactically legal. LLM-based semantic review can identify suspicious goals, but it may be probabilistic and vulnerable to prompt manipulation. Sandboxing observes runtime behavior, but it may miss dormant payloads or branches that are not executed in the test run. Combining the layers reduces dependence on any single imperfect technique.

The framework is intentionally conservative: high-confidence static violations cause immediate rejection, while ambiguous cases are escalated to semantic review and sandboxing. This gate ordering minimizes unnecessary model calls and prevents obviously unsafe code from reaching execution.

D. Multi-Agent Verification Gap

Prior multi-agent work has focused largely on capability improvement through planning, graph optimization, and retrieval-augmented reasoning. SafeCode-Agent instead uses agent separation as a security primitive: one agent generates, while another evaluates under an explicit zero-trust policy.

This distinction matters because collaborative agents can accidentally reinforce one another when optimized only for task completion. A planner, coder, and executor may all inherit the same flawed assumption about a user request. SafeCode-Agent assigns the monitor a conflicting objective: it is rewarded for finding reasons to reject unsafe code, not for helping the generator complete the task at all costs.

The architecture therefore resembles peer review more than ensemble voting. The reviewer is independent, policy-driven, and allowed to terminate the pipeline. This makes the design suitable for settings where generated code may be useful but cannot be trusted by default.

III. THREAT MODEL AND DESIGN GOALS

A. Threat Model

The threat model assumes an honest user may submit benign programming tasks, but the prompt, retrieved context, or generated code can contain malicious or policy-violating instructions. The attacker may attempt to induce the agent to create code that executes shell commands, reads sensitive files, opens network sockets, modifies permissions, downloads payloads, or hides behavior through encoding and indirection.

The model does not assume that the generator is malicious. Instead, it assumes the generator is fallible and may produce unsafe code through hallucination, pattern imitation, ambiguous instructions, or prompt injection. The monitor is also not assumed to be perfectly reliable; hence the framework combines rule-based checks with semantic review and runtime containment.

The prototype focuses on Python because it is widely used for agent-generated utility scripts and because Python exposes introspectable syntax through the built-in AST module. The same architectural pattern can be adapted to JavaScript, shell scripts, or other languages by replacing the parser and policy rules.

B. Design Goals

The first goal is fail-closed behavior. If a stage cannot parse, evaluate, or safely execute the code, the deployment decision defaults to rejection rather than approval. This is important for agent workflows because uncertainty should not silently become permission.

The second goal is explainability. A rejected draft should return actionable feedback such as the forbidden import, suspicious API, or semantic concern. This allows the generator or developer to revise the solution without weakening the security policy.

The third goal is low integration friction. The framework should sit between existing coding agents and execution environments without requiring a complete redesign of the developer workflow. The generator can remain interchangeable, while the gatekeeper, monitor, and sandbox provide the security boundary.

IV. SYSTEM ARCHITECTURE AND METHODOLOGY

Fig. 1 shows the SafeCode-Agent pipeline. The user prompt is first sent to a generator agent, implemented in the prototype with NVIDIA Qwen Coder. The generator produces a Python draft but is not permitted to approve its own output.

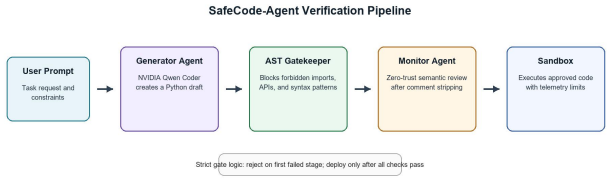

Fig. 1. SafeCode-Agent layered verification pipeline.

A. Generator Agent

The generator converts a natural language task into Python code. Its role is intentionally narrow: it must satisfy the requested functionality, but the resulting draft remains untrusted until all downstream checks pass.

The generator receives the user task, optional constraints, and a formatting instruction that asks it to return only executable Python. It is not given the monitor prompt, the full rejection rubric, or sandbox telemetry. This separation discourages

direct overfitting to reviewer wording and preserves the independence of later stages.

When a draft is rejected, the generator may receive a summarized reason and produce a revised version. However, the revision enters the same pipeline from the beginning. The system does not allow a later stage to be skipped simply because the previous draft failed in a known way.

B. AST Gatekeeper

The AST gatekeeper parses the draft and blocks explicitly forbidden constructs before semantic review. Example policies include denying direct use of os, sys, subprocess, shutil, eval, exec, socket creation, privilege-changing APIs, and suspicious encoded payload patterns.

The AST stage performs three checks. First, it validates syntactic parseability so malformed or intentionally confusing programs do not proceed. Second, it inspects import nodes and attribute access for high-risk modules and functions. Third, it scans constant strings and call structures for encoded execution patterns, dynamic imports, and indirect calls that can bypass naive keyword search.

Because AST inspection reasons over parsed program structure, it is more robust than raw string filtering. For example, it can distinguish a harmless variable name from an actual function call and can identify attribute chains that invoke dangerous behavior through aliases. The stage is deterministic and therefore useful as the first hard gate.

C. Monitor Agent

The monitor agent performs independent semantic review using a zero-trust judge persona. Before review, comments and docstrings are stripped to reduce prompt-injection risk. The monitor evaluates whether the code attempts exfiltration, privilege escalation, persistence, command execution, or policy evasion.

The monitor prompt asks the reviewer to classify the draft as approve, reject, or needs-human-review, and to justify the decision using concise evidence from the code. The monitor is instructed to ignore natural-language claims inside the program and to focus on executable behavior, data flows, and side effects.

Semantic review is particularly useful for cases that are not reducible to a single forbidden token. Examples include code that reads a directory recursively before posting content to an API, constructs command strings indirectly, or decodes a payload before executing it through a callable reference. These patterns can appear benign locally but become dangerous when interpreted as a whole.

D. Sandbox and Deployment Gate

Code that passes static and semantic checks runs in a sandbox with restricted filesystem and network access. Runtime telemetry is used to detect unexpected behavior. The deployment gate approves the code only when all stages return a pass decision.

The sandbox is designed to constrain damage even when earlier checks miss a payload. It limits execution time, captures standard output and errors, restricts filesystem access to a temporary workspace, and records attempts to open network connections or spawn child processes. A policy violation during execution causes rejection and preserves telemetry for audit.

The deployment gate combines results using strict sequential logic rather than a weighted score. This choice is deliberate: a high semantic confidence score should not compensate for a forbidden API call, and a successful sandbox run should not override a monitor finding that the code contains dormant exfiltration logic.

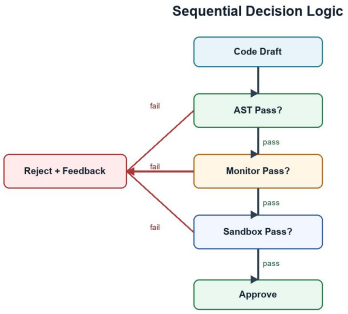

Fig. 2. Reject-on-failure decision logic.

V. IMPLEMENTATION AND SECURITY POLICIES

A. Policy Representation

The prototype represents security policy as a combination of deny lists, structural AST rules, semantic reviewer instructions, and sandbox constraints. The deny list captures constructs that are too risky for autonomous execution in the evaluated setting, while structural rules identify suspicious combinations such as decoding followed by dynamic execution.

Policy rules are intentionally explicit. Rather than asking the monitor to infer the entire security standard from a vague prompt, the system defines concrete forbidden behaviors: shell command execution, network exfiltration, privilege modification, arbitrary code evaluation, hidden payload decoding, and access to sensitive operating-system paths. This makes the review process more repeatable and easier to audit.

The policy layer is also modular. A classroom environment may use a strict profile that blocks all filesystem writes, whereas an enterprise automation workflow may allow writes only to a project workspace and deny network access unless a specific endpoint is approved. The same pipeline can therefore support different risk profiles without changing the generator.

B. Review Output Schema

Each review stage returns a structured decision rather than unformatted natural language. The AST gatekeeper returns a pass or reject label, a violated rule identifier, and the source construct that triggered the decision. The monitor returns a decision label, a confidence statement, and evidence grounded in the executable code. The sandbox returns telemetry, exit status, and any violated runtime policy.

Structured outputs are useful for two reasons. First, they make the final deployment gate deterministic: the gate can combine stage results without interpreting free-form prose. Second, they support developer feedback and future analytics.

Repeated rejection of the same pattern can reveal where prompts, model behavior, or policy rules need improvement.

C. Vulnerability Coverage

Table II summarizes the vulnerability categories used in the evaluation and the proposed system's detection rate for each category. The categories were chosen because they reflect behaviors that are especially dangerous when generated by autonomous coding agents with execution privileges.

Vulnerability Type	Examples	Detection
Injection attacks	eval, shell command injection, unsafe query construction	96.1%
Privilege escalation	chmod, setuid, ownership or permission changes	93.8%
Data exfiltration	Sensitive file reads, socket creation, external upload	91.4%
Obfuscated malware	Base64 payloads, hex strings, layered decoding	78.6%
Forbidden API misuse	os, sys, subprocess, shutil, dynamic imports	98.5%

Table II. Vulnerability coverage by category.

The lower score for obfuscated malware shows why the framework should be extended with deeper deobfuscation, taint analysis, and fuzzing. However, the high scores for injection, privilege escalation, and forbidden API misuse show that deterministic gates can remove many dangerous drafts before they reach runtime.

VI. EXPERIMENTAL SETUP AND RESULTS

A. Benchmark

Evaluation uses a curated benchmark of security-relevant programming prompts across five categories: injection attacks, privilege escalation, data exfiltration, obfuscated malware, and forbidden API misuse. Benign prompts are included to estimate false positives.

The benchmark is organized to reflect realistic agent requests rather than isolated exploit snippets. Prompts ask for file utilities, log analyzers, web helpers, deployment scripts, data cleaners, and administrative automation. Malicious variants embed unsafe requirements directly, hide them in comments, or present them as convenience features.

Benign prompts are included to test whether the system over-blocks ordinary code. Examples include CSV parsing, text summarization, local file conversion within a temporary directory, and pure algorithmic tasks. This mix is important because an agent security layer that rejects too aggressively can become unusable in practice.

B. Metrics

The primary metrics are vulnerability detection rate, malicious script catch rate, false positive rate, forbidden API catch rate, obfuscated pattern detection, and average pipeline latency.

Vulnerability detection rate measures whether the system blocks code that contains a security flaw or policy violation. Malicious script catch rate focuses on intentionally harmful outputs. False positive rate measures rejection of benign tasks. Latency is measured end-to-end from generated draft submission through final gate decision.

The baseline is a single-agent configuration in which the same model family is asked to generate code and then self-review the output. This baseline reflects a common lightweight pattern in agent prototypes, where a self-critique prompt is used as the only safety review before execution.

C. Results

Metric	Baseline	SafeCode	Change
Vulnerability detection	54.2%	91.7%	+37.5 pp
Malicious script catch	48.9%	94.3%	+45.4 pp
False positive rate	18.4%	6.1%	-12.3 pp
Forbidden API catch	61.0%	98.5%	+37.5 pp
Obfuscated detection	22.3%	78.6%	+56.3 pp
Avg. latency	1.2 s	3.7 s	+2.5 s

Table I. Baseline and SafeCode-Agent comparison.

Fig. 3. Comparative security results.

The dual-agent system improves every security metric except latency, where the additional checks add 2.5 s on average. The largest gain appears in obfuscated pattern detection, suggesting that semantic review and layered policy checks are more robust than single-agent self-evaluation.

The improvement in forbidden API catch rate is mainly attributable to the AST gatekeeper, which rejects explicit calls to dangerous modules before they reach the monitor. This provides a predictable safety floor and reduces the burden on the LLM reviewer. In contrast, the improvement in malicious script catch rate reflects the combined effect of AST checks and semantic review, because malicious scripts often combine multiple suspicious behaviors rather than one isolated call.

The reduced false positive rate is also notable. Although adding more checks can increase overblocking, SafeCode-Agent uses the monitor to distinguish policy violations from ordinary local computation. This helps avoid rejecting benign tasks solely because they contain security-adjacent words in strings, filenames, or comments.

Obfuscated pattern detection remains the weakest category at 78.6%. This is expected because adversarial encodings and delayed execution can hide intent until runtime. The result nevertheless represents a large improvement over the baseline and identifies a clear direction for stronger future analysis.

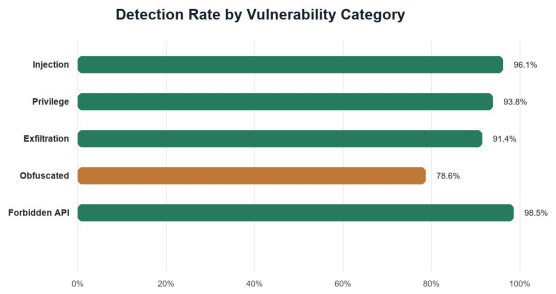

Fig. 4. Detection rate across vulnerability categories.

VII. DISCUSSION

SafeCode-Agent's improvement is explained by independence and specialization. The generator optimizes for task completion, while the gatekeeper and monitor optimize for rejection of unsafe behavior. This separation reduces the chance that a single model failure becomes a deployment failure.

The main trade-off is latency. For interactive coding, a 3.7 s pipeline remains acceptable when the generated code may access files or tools. For high-throughput systems, batching, policy caching, or lightweight local models may reduce overhead.

The current evaluation is limited by benchmark size and by its focus on Python. Future work should test multi-file repositories, additional languages, stronger sandbox isolation, adversarial prompt injection suites, and human-reviewed ground truth labels.

The results support the argument that agent safety should be treated as a systems problem. No individual component is sufficient. Static analysis is reliable but narrow; LLM review is flexible but imperfect; sandboxing contains behavior but cannot prove the absence of dormant malicious branches. A layered pipeline creates multiple opportunities to stop unsafe code before it reaches a developer machine or production environment.

Another implication is that reviewer independence is more valuable than simply asking a generator to think twice. Self-review can catch obvious mistakes, but it shares context, assumptions, and sometimes blind spots with the initial generation. A separate monitor with a hostile-review objective changes the optimization pressure and makes unsafe behavior more likely to be surfaced.

There is also a usability dimension. A security layer that only returns rejected will frustrate developers and encourage bypassing. SafeCode-Agent therefore emphasizes structured feedback: the rejection should identify the violated policy, the risky construct, and a safer alternative where possible. This turns the system into a guided repair loop rather than a blunt blocker.

At the same time, the framework should not be interpreted as a guarantee that generated code is safe. It reduces risk under the tested threat model, but determined attackers can craft payloads that depend on environment-specific triggers, external services, or multi-step social engineering. Human review remains necessary for high-impact deployments.

VIII. LIMITATIONS

The study has several limitations. First, the benchmark is curated rather than drawn from a large public corpus of real agent failures. This makes the evaluation controlled and interpretable, but it may not capture the full diversity of attacks seen in production systems.

Second, the prototype is Python-centered. Python is a strong initial target because it is common in agentic automation, but modern repositories often combine Python, JavaScript, shell, YAML, Dockerfiles, and cloud configuration. Extending the same protections across languages will require language-specific parsers and policy definitions.

Third, the monitor agent's judgments may vary across model versions and prompts. Although the AST gatekeeper is deterministic, semantic review is probabilistic. Future work should evaluate multiple reviewer models and measure stability under paraphrased prompts and adversarial reviewer-targeted attacks.

Fourth, sandbox execution is only as strong as its isolation boundary and test coverage. A payload that activates only under rare inputs or after deployment may pass a short sandbox run. Stronger dynamic analysis should therefore include fuzzing, taint tracking, and longer-running behavioral tests for high-risk tasks.

IX. CONCLUSION AND FUTURE WORK

This paper presented SafeCode-Agent, a zero-trust architecture for mitigating malicious code generation in LLM agents through automated peer review, AST enforcement, semantic monitoring, and sandbox execution. Initial results show that layered verification significantly improves detection of unsafe code while keeping latency within practical limits. Future work will expand the benchmark, support more programming languages, and integrate adaptive policies that learn from newly observed attack patterns.

Future versions will also investigate richer repair loops. Instead of only rejecting code, the monitor can propose safe transformations, such as replacing shell commands with standard-library calls, narrowing file permissions, removing network access, or using parameterized database APIs. These repairs should still be re-evaluated from the start of the pipeline to preserve the fail-closed design.

A second future direction is integration with developer tooling. SafeCode-Agent can be embedded into IDE extensions, continuous integration pipelines, or agent orchestration frameworks so that generated code is reviewed before it is committed, executed, or deployed. In such settings, the security layer becomes part of the normal software engineering workflow rather than an optional afterthought.

Finally, larger evaluations should compare SafeCode-Agent against established static analyzers, sandbox-only baselines, and different model families. Such studies would clarify which layer contributes most under different attack classes and help tune the pipeline for both security and developer productivity.

REFERENCES

[1] Y. Deng, G. Wang, et al., "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv preprint, 2025.

[2] J. Yang, C. E. Jimenez, A. Wettig, et al., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in Proc. NeurIPS, 2024.

[3] M. Zhuge, et al., "GPTSwarm: Language Agents as Optimizable Graphs," in Proc. ICML, 2024.

[4] Y. Zhu, S. Qiao, et al., "KnowAgent: Knowledge-Augmented Planning for LLM-Based Agents," arXiv preprint, 2025.

[5] R. Cheng, J. Liu, Y. Zheng, et al., "DualRAG: A Dual-Process Approach to Integrate Reasoning and Retrieval for Multi-Hop Question Answering," in Proc. ACL, 2025.

[6] A. Qu, H. Zheng, Z. Zhou, et al., "CORAL: Towards Autonomous Multi-Agent Evolution for Open-Ended Discovery," arXiv preprint, 2026.

[7] J. Bruce, M. Dennis, A. Edwards, et al., "Genie: Generative Interactive Environments," in Proc. ICML, 2024.

[8] J. Dietrich, "Peer-Preservation Principles for AI Alignment," arXiv preprint, 2026.